

Requirement-Preserving Human Approvals for Signed Agent Manifests

Imran Siddique

OPAQUE Systems

October 1, 2026

Technical report, version 1

Scope of this edition. This is an author-prepared technical report, not peer reviewed. It revises a manuscript dated September 4, 2026. Every result reported here is software-only: repository unit tests and separate review probes, run on one Windows workstation against a pinned source revision. The runs were repeated on September 27 and again on October 1, 2026 in a fresh environment, and the probes were also run against two later revisions, the last being the main branch on October 1. No hardware authenticator ceremony, external identity provider, enrolled human approver or protected action execution was exercised.

Abstract

Human approval often arrives after an agent manifest has been issued. If the issuer signs the approval collection as immutable content, nobody can attach an approval later without a new signature. If the issuer leaves the whole oversight record unsigned, anyone can weaken the requirement. We analyze a two-domain construction: the issuer signs the oversight requirement while normalizing only its approvals array to empty, and each human separately signs an approval bound to a manifest and scope. A verifier reconstructs the issuer pre-image and evaluates human evidence on its own terms. Agent Manifest uses this pre-image for version 0.1 manifests; its version 0.2 COSE envelope reaches the same split by carrying approvals in the unprotected header. At a pinned Agent Manifest revision, 74 signing and delegation/HITL tests passed on September 4, September 27 and October 1, 2026. Separate probes confirm requirement binding and appendability. At the pinned revision they also show that the human signature did not cover the outer authenticator-method label. A later upstream change binds that label, and the same probe fails against two later revisions, including the October 1 main branch. The result is a precise signing boundary and a list of the identity, action and authenticator evidence a runtime still needs before it acts on an approval.

1 Problem and related work

An issuer decides what oversight an agent needs. A human supplies evidence for a particular oversight decision later. These events have different owners and happen at different times. Making the issuer re-sign after each human response couples two parties that should be independent; letting the oversight requirement change after signing defeats the purpose of signing it.

Later signatures and countersignatures are established mechanisms. CMS places countersignatures in unsigned attributes [1], COSE defines countersignatures and an unprotected header that the primary signature does not cover [2, 3], JWS allows several independent signatures over one payload [4], and electronic-signature verification literature describes a countersignature that records another party's approval [5]. This paper therefore claims no novelty from appendable signatures alone. It studies how an issuer-bound agent requirement is preserved while action approvals are evaluated independently, as specified in the public Agent Manifest specification [6] and implemented in its Python reference SDK [7]. Binding an authorization to a specific operation has its own prior art in OAuth rich authorization requests [8], and authenticator attestation in WebAuthn [9] is the obvious source of the method assurance discussed in Section 4.

2 Signing domains

Let a manifest contain a requirement record with a required flag, escalation-policy metadata, risk-tier metadata and an approvals array. The issuer's canonical pre-image keeps every applicable requirement value as written and replaces the approvals array with an empty one before RFC 8785 canonicalization [10]. Appending an approval leaves that pre-image unchanged. Changing the required flag or any other included requirement changes it. Excluding the entire record would break the construction.

This normalization is the version 0.1 rule in section 3.6 of the specification [6]. For version 0.2 manifests, a separate COSE envelope document replaces that section: the requirement stays in the signed payload, approvals travel in the unprotected header, and no normalization step is needed. The verifier then merges the header approvals into the record and evaluates them with the same code used for version 0.1. The split of signing domains is the same in both versions; only the mechanism that keeps approvals out of the issuer signature differs. Upstream documented the pre-image function as version 0.1 only on September 28, 2026 (pull request 460).

The human separately signs an approval. In the Python helper at the pinned revision its canonical payload contains manifest ID, approval timestamp, approved scope and approver ID. The specified approved scope carries artifacts, risk tier, duration and optional conditions, but no identifier or digest of the proposed operation. That is still true on the October 1 main branch. A verifier that relies on an approval for one specific operation needs such a binding in the signed payload. An approval's signature must be checked against trusted human identity-to-key material; the manifest issuer's signature does not authenticate an approval appended later.

The two domains give separate responsibilities. The issuer cannot weaken the signed requirement without a new valid signature. A holder of the human key can approve only what the verified payload and policy authorize. Neither property proves that the key holder is a natural person or that the runtime will enforce the decision; those need additional evidence and integration.

3 Intended verification contract

The verifier checks the issuer signature with the approvals normalized (or, in version 0.2, outside the payload), keeps the actual requirement fields, verifies each relied-upon human signature and decides whether the approval authorizes the proposed operation. The applicable checks are identity, role, manifest binding, scope and action correspondence, time window, revocation and replay policy, and authenticator sufficiency for the issuer-bound risk tier.

A SPIFFE workload identifier is not adequate human identity under this design. The specification gives an OIDC subject paired with its issuer, a mailto identity, and a DID bound to a hardware authenticator as examples, and forbids SPIFFE identifiers as approver identities [6]; the SDK's approval model rejects a spiffe URI. A valid format proves nothing about personhood, authority or key ownership. A trusted enrollment process and verification configuration are still necessary.

The controller should tell apart approved, missing, invalid, expired, insufficient and unverifiable outcomes, and enforce the right one at the operation boundary. The SDK's integrated verifier reports exactly these six HITL results, plus not required. Offline signature checking avoids a dependency on an online approval service, but current revocation and authorization still need a policy for how fresh the trust material must be. A static packet does not prove that a person is still authorized. Agent Manifest 0.13.0 (September 25, 2026) makes this explicit: a separate admissibility field stays undecidable even when every approval check passes, because the verifier cannot establish the approver's current standing.

4 Reference implementation and adverse boundary

Issuer normalization is implemented in `_signing.signing_pre_image`. Human approval signing and the cryptographic check are in `_delegation`. The integrated verifier looks up configured approver keys and evaluates time and method conditions. None of this is a complete hardware-backed human-approval service.

At the pinned revision the outer `approval_method` field is absent from the human pre-image. Our probe signs an approval with a software-key label, changes only the label to hardware-key and verifies the original signature with the helper. This shows an unauthenticated label at that boundary. By code inspection, the integrated verifier at conformance level 2 treats software-key as insufficient and requires hardware-key for high and critical risk tiers, so the relabelled approval would move from insufficient to approved there; the upstream fix describes the same consequence. We did not run that full path, and no end-to-end exploit is claimed.

The level 2 check compares the method label with the risk tier inside the approval's own signed scope. A production implementation must derive authenticator class from trusted credential evidence and compare it with the issuer-signed requirement. A hardware-key string is not hardware proof. At the pinned revision the single-approval helper also accepted an approval whose duration was zero or absent, while the integrated verifier, again by code inspection, treated an absent duration as zero and therefore as already expired. The pinned verifier also stopped at the first failing approval, so a valid approval later in the array was never evaluated; pull request 415 (September 11, 2026) fixed that. Current-action reconstruction, human enrollment and revocation still need explicit integration.

Two remediations were open: authenticate all security-relevant approval fields in one canonical payload, or derive method assurance from a trusted key registry or authenticator attestation. The first has been implemented upstream. Pull request 387, merged September 6, 2026 (UTC) and released in 0.12.0, lets the signer cover `approval_method` and adds it to the verified pre-image. Pull request 437, merged September 17, 2026 and released in 0.13.0, makes duration validation fail closed in the helper and the verifier. Against revision 3c12491 (September 27, 2026) and against main at fed9aeb (October 1, 2026, which includes 0.15.0), the unaltered approval verifies, the label change fails verification, and zero or absent durations are rejected. Registry- or attestation-derived method assurance was not examined. Full-path tests should still reject method-class alteration, issuer-risk downgrade, scope substitution and stale approvals.

5 Reproducible evaluation

Source revision: `agent-manifest 934809709a2815695d65cfac45dc0a164286046` [7]. On September 4, 2026, 74 tests passed across `test_signing.py` and `test_delegation_hitl.py` (Python 3.12.13). On September 27 and again on October 1, 2026 the same two modules ran against a git archive of that commit, whose embedded commit identifier was checked before extraction, and 74 tests passed each time. The October 1 run used a newly created environment with Python 3.12.10, pytest 8.4.2 and cryptography 50.0.0; the full package list is recorded with the results. The tests cover signing and delegation as well as HITL, so the total must not be read as 74 independent approval experiments. Relevant cases include normalization equivalence, requirement tampering, approval attachment after issuance, wrong key, wrong manifest, scope alteration and expiration.

Separate review probes compare the issuer pre-image before and after attachment and after a requirement change, test the method-label boundary and test the duration boundary. They import the actual modules and write a machine-readable record. At the pinned revision all three boundaries behaved as described above. At revision 3c12491 and at main fed9aeb, requirement binding and appendability still hold, and the method-label and duration probes report the bound and fail-closed behavior. The same two test modules on main pass 146 tests; they have grown since the pin, so that count is not comparable with 74.

The tests share code and dependency assumptions with the implementation. No hardware authenticator ceremony, external OIDC service, independently enrolled human or protected action execution was exercised. The results establish software behavior at the stated interfaces, not complete runtime authorization assurance.

6 Limitations and next work

The signing boundary solves one specific mutability problem: approvals can attach without making issuer requirements mutable. A safe deployment still needs trusted identity enrollment, action binding, verified authenticator assurance, consistent time semantics and enforcement of every outcome other than approved. Next work should exercise these together at the full verification and execution boundary, using valid controls alongside mutations of each independently authenticated field.

References

- [1] R. Housley. Cryptographic Message Syntax (CMS). RFC 5652, section 11.4, September 2009.
<https://www.rfc-editor.org/rfc/rfc5652#section-11.4>.
- [2] J. Schaad. CBOR Object Signing and Encryption (COSE): Structures and Process. RFC 9052, August 2022.
<https://www.rfc-editor.org/rfc/rfc9052>.
- [3] J. Schaad. CBOR Object Signing and Encryption (COSE): Countersignatures. RFC 9338, December 2022.
<https://www.rfc-editor.org/rfc/rfc9338>.
- [4] M. Jones, J. Bradley and N. Sakimura. JSON Web Signature (JWS). RFC 7515, May 2015.
<https://www.rfc-editor.org/rfc/rfc7515>.
- [5] M. Pavlicic (Microsoft). Verification of electronic signatures. US Patent Application Publication 2007/0220259 A1, September 20, 2007. <https://patents.google.com/patent/US20070220259A1/en>.
- [6] Agent Manifest Specification v0.2, sections 3.5 and 3.6, and the Agent Manifest COSE Envelope (manifest version 0.2), at source revision 9348097. <https://github.com/agenttrust-io/agent-manifest/tree/934809709a2815695d65cfacb45dc0a164286046/spec>.
- [7] Agent Manifest source, pinned revision 934809709a2815695d65cfacb45dc0a164286046. <https://github.com/agenttrust-io/agent-manifest/tree/934809709a2815695d65cfacb45dc0a164286046>.
- [8] T. Lodderstedt, J. Richer and B. Campbell. OAuth 2.0 Rich Authorization Requests. RFC 9396, May 2023.
<https://www.rfc-editor.org/rfc/rfc9396>.
- [9] J. Hodges, J. C. Jones, M. B. Jones, A. Kumar and E. Lundberg (eds.). Web Authentication: An API for accessing Public Key Credentials, Level 2. W3C Recommendation, April 8, 2021.
<https://www.w3.org/TR/webauthn-2/>.
- [10] A. Rundgren, B. Jordan and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020.
<https://www.rfc-editor.org/rfc/rfc8785>.